

The Impact of Static Analysis Depth Limit on Semantic Conflict Detection

Anonymous Author(s)

Abstract

Collaborative software development requires periodic code integration of independent contributions into a shared codebase, a process that may introduce dynamic semantic conflicts (behavioral interference between changes not captured by textual merge tools). Static-analysis-based approaches for detecting such conflicts rely on configuration parameters that affect both detectability accuracy and computational cost. A key parameter is the method traversal depth limit, which determines how deeply chains of method calls are followed when tracking interference. In this paper, we analyze how varying the depth limit affects both the number of reported interferences and the execution performance of the Overriding Assignment (OA) analysis for interference detection. We evaluate it on two datasets from previous studies: the MergeDataset, a manually curated benchmark, and the RefDataset, a broader and more diverse dataset covering a wider range of project sizes and merge patterns. Both datasets comprise Java open-source projects from GitHub, and our findings are consistent across them. We execute the analyses limiting the depth to 5, 10, and 20 method calls. Our results show that increasing the depth limit does not consistently improve detection: the relationship is non-monotonic. The gain is most notable when moving from depth 5 to 10, where the number of newly reported scenarios increases. Beyond depth 10, however, the number of detected scenarios and total interferences both decrease considerably. These gains come at a measurable cost: average analysis time and memory consumption increase noticeably at greater depths, with diminishing returns. Nevertheless, accuracy and precision tend to improve from depth 5 to 10, but degrade when moving further to depth 20.

Keywords

Merge Tools, Semantic Conflict, Static Analysis, Depth Tuning, Pointer Analysis

1 Introduction

Software projects depend on developers regularly merging their work, though resolving conflicts remains a significant challenge in practice [5, 17, 19]. In addition to textual clashes, reported by tools such as `diff3` [12], subtler problems arise when parallel modifications affect program behavior in unexpected ways. These dynamic semantic conflicts arise when a merge combines code from separate branches in ways that inadvertently alter the behavior of existing changes or the original program. Early detection is therefore critical to integration quality [9, 18].

Detecting this class of conflicts requires reasoning about program behavior. Static analyses for detecting semantic

conflicts were proposed early on [11], but their reliance on multiple versions of system dependence graphs makes them impractical even for codebases of moderate size [3]. To address this scalability gap, Barbosa et al. [3] introduced *Overriding Assignment* (OA) analysis, a lightweight static analysis [7] that identifies interference by checking whether changes from two developers write to the same state element (variable, field, or parameter) without any base-version write in between. Their interprocedural OA (iOA) variant traverses called methods recursively up to a configurable *depth limit*; in their evaluation, however, this limit was fixed at 5 without examining how the choice affects interference recall or execution cost.

The depth limit is a critical, yet underexplored, configuration parameter of iOA. When the limit is too shallow, the analysis halts before reaching write operations buried in called methods, missing interferences and reducing recall. When it is too deep, the traversal covers an increasingly large portion of the call graph, raising execution time and memory to potentially prohibitive levels. There is thus a need for principled guidance on how to configure it.

In this paper, we address this gap by empirically investigating how varying the depth limit (set to 5, 10, and 20) affects both the detectability of OA interferences and the execution performance of the analysis, using two datasets from prior work. The *MergeDataset* [20] contains 99 three-way merge experimental units comprising methods mutually modified by both developers across Java open-source GitHub projects, with manual ground-truth interference labels. The *RefDataset* [16] is a larger collection of 907 experimental units (of which 846 pass an additional build filter, described in Section 4.1) mined from a broader set of Java projects, covering a wider range of project sizes and merge patterns, without per-scenario manual ground truth. Both datasets comprise real merge scenarios¹ from complete projects at different scales and validation levels.

To investigate this trade-off, we run iOA using the SPARK pointer-analysis framework (built on top of Soot) [13], following the most prominent configuration in Barbosa et al. [2].² For each depth limit, we execute the analysis ten times per experimental unit to gather stable performance measurements. Because timeouts are a known concern in prior work [2], we adopt a *partial-result strategy* that caps analysis at the point of timeout explosion, allowing us to include partial results up to that boundary rather than discarding the entire scenario.

From these executions we collect execution time both at the dataset level and per scenario, as well as the number

¹A merge scenario is a three-way merge in which both developers modified the same methods.

²SPARK restricts callee resolution to classes actually instantiated via `new`, pruning the call graph to realistically executable paths [2].

of scenarios for which iOA reports at least one interference, and the overall set of detected interferences. For the Merge-Dataset, we further evaluate the behavior of newly reported scenarios at each depth, computing precision, recall, accuracy, and F1-score to enable a direct comparison across depth configurations.

The main contributions of this paper are: (i) an empirical evaluation of the impact of depth tuning on OA interference detectability; (ii) a structural characterization of interference locality and call-depth distribution across the studied datasets; (iii) a characterization of the corresponding performance costs across two datasets of distinct scales; (iv) a practical recommendation that depth values close to 10 represent a suitable configuration for the analyzed datasets, reducing the share of entirely missed scenarios while remaining computationally feasible for most scenarios; and (v) a *partial-result strategy* that caps analysis at timeout boundaries, allowing timed-out scenarios to still contribute detections rather than being discarded entirely.

2 Motivating Example

To demonstrate how the depth limit affects the detection of interferences, consider the merge scenario shown in Figure 1.



Figure 1: Merged ConfigManager code from a three-way merge.

It shows the Java code of the ConfigManager class resulting from a three-way merge in which two developers modified the

base version in parallel. Developer Left modified the existing values in loadClientDefaults, changing timeout from 60 to 45 and retries from 3 to 2. Developer Right introduced a call to tuneForConnectionReuse in the constructor and provided its implementation, which eventually sets timeout = 20 inside applyKeepAlivePolicy.

This constitutes an OA interference because Developer Right's write operation to timeout that is defined inside applyKeepAlivePolicy overrides Developer Left's write operation timeout = 45 with no textual conflict. The merged program silently applies a 20-second timeout instead of the 45-second value Developer Left intended. No textual conflict is reported because the two changes do not touch the same or consecutive lines.

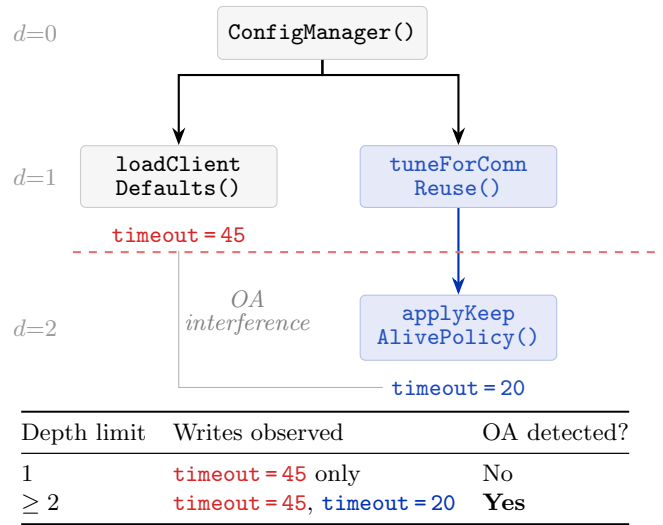


Figure 2: Depth-limited analysis traversal of ConfigManager.

To detect semantic interference in merged code, iOA traverses method calls up to a configurable *depth limit* (d), recording every write to state elements it encounters along the way.³ Figure 2 illustrates how that limit affects what the analysis can observe. Starting at the constructor ($d=0$), it follows each call one level deeper per step. At depth 1 it enters loadClientDefaults and records timeout = 45 (Developer Left). Besides that, it also enters tuneForConnectionReuse but, with limit 1, halts without descending into the method: applyKeepAlivePolicy. Developer Right's write is invisible, so no OA interference is detected. Raising the limit to 2 brings applyKeepAlivePolicy into the analysis scope: both writes to timeout are now observed with no base-version write between them, and the OA interference is correctly detected.

³State elements refer to program entities that hold or represent the state of a program during its execution. These include global variables, object fields, system files and streams, variables that store method return values, raised exceptions, and any other entities that maintain state information across different execution contexts.

This example illustrates the core trade-off: deeper traversal reaches more write sites, reducing the chance a genuine interference goes undetected, but at the cost of more analysis work and longer execution times.

In practice, some scenarios escalate into exponential growth of analysis states and trigger a *timeout explosion*: a single subtree of the call graph can expand into exponentially many paths, exhausting the analysis budget before other subtrees are visited; an interference located in a different branch may never be reached despite falling within the declared depth limit. Like the depth limit, the per-scenario timeout is a configurable parameter: without it, analysis could run indefinitely on pathological call graphs. Taken together, timeout explosion and depth-limit sensitivity make the choice of depth limit a trade-off between *detectability* and *computational cost*. Understanding how different depth configurations shift this balance, across precision, recall, and execution time, motivates our study.

3 Static Analysis

An OA interference occurs, for example, when a merge integrates two developers’ changes that write to the same state element with no base-version write in between, causing one write to silently override the other. In merge scenarios, such interferences may indicate behavioral conflicts in which the merged program exhibits behavior neither developer individually intended. Static analysis can detect these interferences as an approximation to finding semantic conflicts.

One approach to this problem, which we focus on, is the *Overriding Assignment* analysis [3], a lightweight static analysis that detects interferences in merged code. The OA analysis scans the merged code, which is annotated so that each instruction carries the identity of the developer who introduced it or marks it as base code. The algorithm maintains two definition sets, S_L and S_R , one per developer. As it scans the merged code, each write instruction adds its target state element to the corresponding set; an OA interference is detected as soon as a write targets an element already present in the *other* set, since this means both developers modified the same state without a base write in between, exactly the overriding condition. When the analysis encounters a base-version write (a write not marked as left or right), it removes that state element from both S_L and S_R , resetting the interference boundary and clearing any prior interference recorded for that element.

The iOA analysis extends this scan across method-call boundaries: starting from the annotated entry method modified by both developers, the iOA follows each call recursively up to a configurable *depth limit*, applying the same S_L/S_R logic in each visited callee. The call graph that governs which callees to visit is built using SPARK [13].

A compounding factor is that virtual dispatch introduces sibling subtrees: both CHA (Class Hierarchy Analysis, a conservative algorithm that includes all subtypes as potential callees) and SPARK include multiple implementations as potential callees of a call site. The analysis explores these

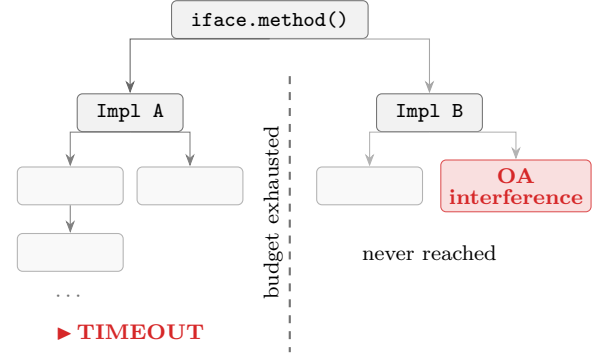


Figure 3: Call-graph timeout explosion.

sequentially; if one subtree exhausts the time budget, the remaining siblings go unexplored, silently missing any interferences they contain. Figure 3 illustrates this: *Impl A*’s large subtree consumes the budget before *Impl B* is visited, so *Impl B*’s OA interference is missed (a false negative caused not by insufficient depth but by budget exhaustion in a sibling subtree).

4 Study Setup

We investigate the impact of varying the iOA depth limit on interference detection and performance, evaluating changes in false positives (FP: reported when none exists) and false negatives (FN: missed actual interferences) across depth configurations, alongside execution time and memory usage. We address the following research questions:

- RQ1.** How does the depth limit affect the number of detected interferences in merge scenarios?
- RQ2.** How is accuracy affected by depth limitation?
- RQ3.** How do different depth configurations affect computational performance?

Together, these questions provide insights into the structural properties of OA interferences across depth configurations, including how interferences distribute across call depths and whether they occur within the same method, class, or across class boundaries (characteristics we map empirically and present as part of our results). This enables practitioners and researchers to make informed decisions when tuning the depth parameter to achieve better results in their semantic conflict detection analyses (Figure 4).

4.1 Dataset

We use two datasets from prior work. The *MergeDataset* [20] comprises merge scenarios (base, left, right, and merge versions) in which two contributors changed the same methods or constructors. It provides a manually labeled set of 99 experimental units associated with 54 merge scenarios from 32 open-source Java systems. As a merge scenario might have more than one declaration that was independently changed by both

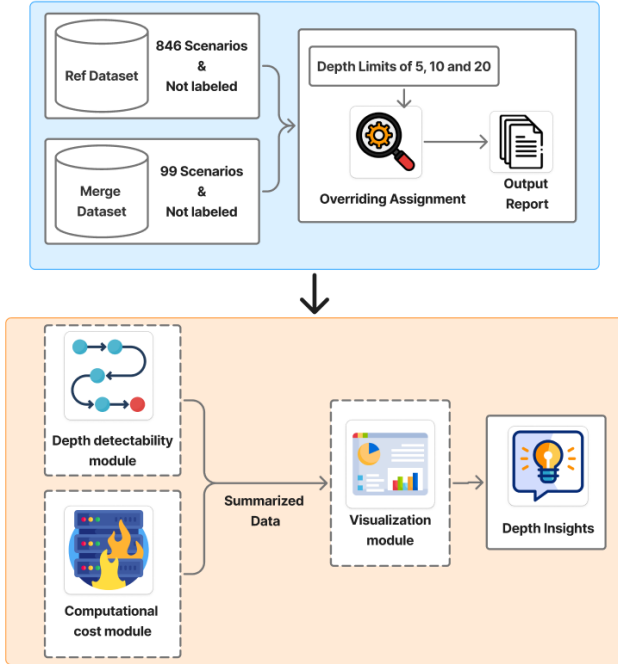


Figure 4: Overview of the study methodology.

developers, we consider pairs (scenario, declaration) as our experimental units. The MergeDataset scenarios [4, 20, 21, 23] have been used in previous studies on semantic conflicts.

Similarly, the RefDataset [16] contains methods or constructors modified by two contributors. The RefDataset consists of 907 experimental units from 295 projects, selected and balanced satisfying the following criteria: GitHub’s greatest hits repositories with at least 10 stars, using Maven or Gradle for build automation, and with the latest main branch commit successfully compiling and passing tests within 30 minutes on JDK 8, 11, or 17. This ensures a higher likelihood of automated `.jar` file generation, which is needed for running the analyses. Lastly, we apply an additional filter to confirm that the target class and method changed by two developers are included in the final build’s `.jar` file. After this filter, 846 experimental units remain and form the dataset used in this study.

For each scenario in both datasets, we obtain the JAR file of the merge commit version without external dependencies. This reduces the amount of code analyzed, improving execution time, although it may decrease accuracy; this trade-off is promising for practical uses of static analysis in detecting interference, and thus we adopt this design. Along with this, we collect the line numbers corresponding to the changes introduced by the Left and Right sides. We focus on cases where two developers modified the same method, as such scenarios are more prone to behavioral interference.

4.2 Study Procedures

For RQ1, our goal is to understand how the depth limit shapes the composition of detected interferences within merge scenarios, that is, how many interferences are found, at which depths they surface, and how they are distributed across scenarios.

To address this, we execute iOA using SPARK at depth limits of 5, 10, and 20 over both the *MergeDataset* and *RefDataset* (our labeled dataset and its extension, respectively). We select these depth values to investigate the impact of increasing traversal depth: depth 5 represents the default established in prior work [3], while depths 10 and 20 allow us to explore how deeper method call traversals affect interference detection. We choose the SPARK pointer-analysis framework as our call-graph construction strategy, following the most prominent configuration evaluated in prior work on pointer analysis refinements to iOA [2]. For each configuration, we monitor the analysis output per scenario, tracking not only whether iOA detects an interference, but also the specific depth at which each interference occurs, along with the classes and methods involved. Since a single experimental unit may contain multiple interferences, we also examine the composition of interferences within each scenario, analyzing how individual interferences relate to one another and how that composition changes across depth configurations.

For RQ2, our goal is to assess how accurately each depth configuration identifies true semantic conflicts, understanding whether deeper traversal improves or degrades the quality of reported results. To measure accuracy, we restrict this analysis to the *MergeDataset*, which provides per-scenario ground truth based on the locally observable interference (LOI) criterion [3]. We compute precision, recall, and F1-score for each depth configuration by comparing the analysis output against this ground truth.

For RQ3, we characterize the computational cost of each depth configuration by monitoring execution time (total and per-scenario, as reported by iOA) and memory utilization (sampled at one-second intervals).

4.3 Experimental Setup

All experiments use Soot 4.5.0 with default settings from Barbosa et al. [2]. We preserve original variable names and line numbers in bytecode, exclude standard Java libraries (except basic classes), and analyze only project code. Analyses run in 10 Docker containers on Ubuntu 22.04 LTS, each with 20 GB memory and 4 CPU cores. We classify a scenario as a positive result when iOA detects at least one OA interference before the analysis either completes or reaches the predefined timeout. Given the timeout explosion behavior described in Section 3, we impose a time limit of 20 minutes per scenario. Rather than discarding scenarios that hit this limit, we adopt a *partial-result strategy*: the algorithm caps the analysis at the timeout boundary and records whatever interference the traversal reaches up to that point, so timed-out scenarios still contribute to detection counts and accuracy evaluation.

The 20-minute limit balances allowing natural termination while preventing indefinite execution on pathological inputs. We repeat computational cost analyses 10 times with low variability across runs. To ensure reproducibility, all experimental artifacts are publicly available [1].

5 Results and Discussion

5.1 Dataset Characteristics: Interference Locality and Symmetry

Since each depth configuration may surface different interferences (deeper traversals can reach write sites invisible at shallower depths), understanding the structural characteristics of interferences in our datasets provides context for interpreting the RQ findings.

We characterize interference composition by analyzing whether interferences in a given scenario occur within the same class and method across the left and right changes. Table 1 shows the percentage of scenarios in which more than 99% of their interferences share the same class and method across both developers’ changes.

Table 1: Percentage of scenarios where >99% of interferences share the same class and method across both changes.

Depth	<i>MergeDataset</i>		<i>RefDataset</i>	
	Class (%)	Method (%)	Class (%)	Method (%)
5	100.0	93.3	99.5	94.8
10	100.0	85.7	99.5	94.5
20	100.0	92.3	98.5	93.7

Both datasets are composed of interferences that occur within the same class and method across the left and right changes, with class locality reaching nearly 100% in all configurations and method locality consistently above 85%. This finding reveals that most interferences in our datasets are localized to the same methods modified by both developers.

We also characterize interference symmetry by computing the *depth diff*, the absolute difference between the call depth at which the left interference and the right interference occur. In the motivating example, Developer Left’s write occurs at depth 1 (`loadClientDefaults`) and Developer Right’s at depth 2 (`applyKeepAlivePolicy`), yielding a depth diff of 1. Table 2 reports, for each depth configuration, the percentage of scenarios in which at least 99% of their interferences have a depth diff of zero, meaning both sides reach the conflicting state from the same call depth.

In most cases, both sides of the merge access the conflicting state from the same call depth. This suggests that, in these datasets, OA interferences tend to be symmetric: both branches modify the same resource through call paths of equivalent length. The pattern holds across depth configurations and both datasets, suggesting it is not an artifact of a particular depth setting.

Table 2: Percentage of scenarios where $\geq 99\%$ of interferences have depth diff = 0 (left and right interference at the same call depth), with scenario counts.

Depth	<i>MergeDataset</i>	<i>RefDataset</i>
5	66.7	83.6
10	60.0	78.4
20	69.2	77.1

5.2 RQ1: Interference Detection Across Depth Configurations

Table 3 shows the number of scenarios for which iOA using SPARK reported at least one interference (detected) and those for which no interference was reported (not detected), for each depth configuration across both datasets. Note that a scenario is counted as detected when at least one OA interference is found within it; Table 4 separately counts the total number of individual interferences reported, so one detected scenario can contribute multiple interferences.

Table 3: Number of experimental units with at least one detected OA interference vs. not detected, per depth.

Depth	<i>MergeDataset</i>		<i>RefDataset</i>	
	Detected	Not Detected	Detected	Not Detected
5	20	79	256	590
10	18	81	263	583
20	16	83	247	599

The results reveal a non-monotonic relationship between depth and detection. Increasing the depth from 5 to 10 allows the analysis to reach write operations buried deeper in call chains, bringing previously unreachable write sites into scope and raising the number of detected scenarios in the *RefDataset*. However, pushing the depth further to 20 reverses this trend: the number of detected scenarios decreases in both datasets. This is consistent with the timeout explosion behavior described in Section 3: deeper traversals increase the probability of exhausting the per-scenario time budget before the analysis completes, causing scenarios that would have been detected at shallower depths to time out and produce no result. This tension illustrates that depth tuning requires finding a balance: too shallow a limit misses interferences hidden in deep call chains, while too deep a limit risks exceeding the computational budget and silently dropping detections altogether. Notably, in the *MergeDataset*, this timeout-driven suppression is already visible at depth 10, where detected scenarios drop from 20 to 18: even a moderate depth increase can trigger timeout explosions in scenarios that would otherwise be detectable.

Table 4 complements this view by showing the total number of individual interferences reported across all scenarios, rather than just the scenario count.

Table 4: Total number of OA interferences reported per depth configuration and dataset.

Depth	<i>MergeDataset</i>	<i>RefDataset</i>
5	22	468
10	31	477
20	19	423

To understand why total interference counts can increase even when fewer scenarios are detected, Table 5 shows the distribution of scenarios by their number of interferences, grouped into depth ranges. Percentages are relative to the number of detected scenarios at each configuration.

Table 5: Distribution of detected scenarios by number of interferences per scenario (% of detected scenarios).

Configuration	Interferences per scenario				
	0–1	2–3	4–6	7–10	11+
depth 5, <i>Merge</i>	26.7%	6.6%	66.7%	—	—
depth 10, <i>Merge</i>	26.7%	6.6%	46.7%	20.0%	—
depth 20, <i>Merge</i>	30.8%	7.7%	53.8%	7.7%	—
depth 5, <i>Ref</i>	36.2%	37.5%	26.3%	—	—
depth 10, <i>Ref</i>	35.3%	30.7%	22.1%	11.9%	—
depth 20, <i>Ref</i>	37.1%	30.2%	22.0%	6.8%	3.9%

The distribution reveals a key mechanism behind the total interference counts in Table 4. At depth 5, all detected scenarios are confined to the 0–6 interference range. As the depth increases to 10 and 20, a portion of scenarios shifts into higher ranges (7–10 and beyond), meaning that the scenarios which do complete without hitting the timeout are able to reach more interferences per run. This explains why depth 10 yields more total interferences than depth 5 despite detecting fewer scenarios in the *MergeDataset* (a reduction driven by two false-positive scenarios timing out, as established in RQ2): the scenarios unaffected by timeout explosion compensate with a richer set of detected interferences. At depth 20 the timeout pressure outweighs this gain, reducing both detected scenarios and total interferences. Together, the two tables show that the effective detection yield of a depth configuration depends not only on how many scenarios complete, but on how many interferences each completing scenario is able to reach.

To isolate the effect of the depth limit itself, *independent of timeout explosion*, we fix depth 20 as a reference and measure, for each shallower depth value, the percentage of *scenarios lost*: scenarios in which every detected interference occurs at a depth greater than the candidate limit, meaning a shallower setting would have reported nothing for that scenario at all. Figure 5 shows this loss curve from depth 5 to 14. This analysis assumes depth 20 is sufficient to surface all reachable interferences; interferences beyond depth 20 would be missed at all evaluated configurations and are outside our study’s scope.

The figure shows that shallow depth limits impose a meaningful detection penalty purely from depth restriction, before any timeout effect is considered. At depth 5, the default used by Barbosa et al. [3], 14.6% of *RefDataset* scenarios and 7.7% of *MergeDataset* scenarios are entirely missed because all their interferences reside deeper in the call graph. The loss decreases sharply as depth grows: at depth 10, scenario loss drops to 3.4% for *RefDataset* and 0% for *MergeDataset*, indicating that most detectable interferences are accessible within 10 levels of call-chain traversal. Beyond depth 10, the *MergeDataset* shows no further losses, and *RefDataset* losses fall to 0.5% or below, confirming that depth 10 effectively captures nearly all reachable interferences while avoiding the heavier timeout exposure of depth 20.

5.3 RQ2: Accuracy Across Depth Configurations

To evaluate accuracy, we use the *MergeDataset*, which provides per-scenario ground truth (see Section 4.2). Each detected scenario is classified as a true or false positive by comparing the analysis output against that ground truth, allowing us to compute precision, recall, and F1-score for each depth configuration. Table 6 reports the classification summary for each depth.

Table 6: Classification summary for *MergeDataset* across depth configurations

	Depth 5	Depth 10	Depth 20
TP	5	5	4
FP	15	13	12
TN	55	57	58
FN	24	24	25

iOA using SPARK is one of several approaches for detecting semantic conflicts, and the *MergeDataset* contains scenarios that are inherently outside its detection scope regardless of depth. Absolute metric values therefore reflect both technique limitations and dataset composition; the meaningful signal lies in how metrics *shift* across depth configurations.

Several patterns emerge from Table 6. First, the number of true positives remains stable from depth 5 to 10 (TP = 5), meaning the additional call-chain reach gained by going deeper does not convert any missed true interference into a detection: the newly reachable write sites at depth 10 correspond to false positives, not ground-truth interferences. Second, the reduction in false positives from depth 5 to 10 (FP: 15 → 13) is best explained by the timeout explosion effect described in Section 3: scenarios that were previously reported as false positives at depth 5 now exhaust the time budget before producing any output, causing them to silently move from the detected set to TN. This FP reduction is therefore coincidental (an artifact of the timeout suppressing output, not of the analysis becoming more precise), which tempers the apparent accuracy gain at depth 10. This is confirmed by the corresponding TN increase (55 → 57), which

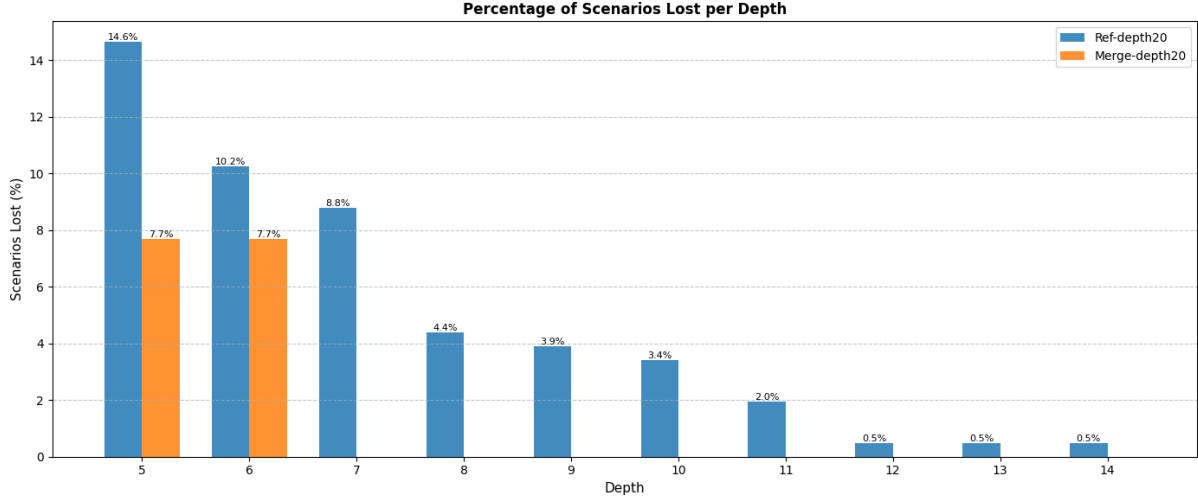


Figure 5: Percentage of scenarios lost per depth configuration, with depth 20 fixed as reference to isolate the effect of call-graph traversal depth on detection.

accounts for exactly the two FP scenarios that dropped out. The FN count remains unchanged (24), consistent with the observation from Figure 5 that depth 10 loses no true scenarios relative to the depth-20 baseline. Going from depth 10 to 20, the pattern shifts: one TP is lost ($5 \rightarrow 4$) and FN grows by one ($24 \rightarrow 25$), meaning a scenario with a genuine interference now times out before that interference is reached, a direct instance of the timeout explosion silencing a true detection. The continued FP decrease ($13 \rightarrow 12$) follows the same mechanism: one more false-positive scenario exhausts its budget and exits the detected set. Together, these shifts indicate that accuracy changes across depths are driven less by the analysis becoming inherently more or less precise, and more by the timeout explosion selectively removing scenarios from the result set as traversal depth grows.

Table 7 makes these effects explicit through the derived metrics.

Table 7: Accuracy metrics for *MergeDataset* across depth configurations. Values in parentheses show relative change from the previous depth.

	Depth 5	Depth 10	Depth 20
Precision	0.25	0.28 (+11.2%)	0.25 (−10.1%)
Recall	0.17	0.17 (0.0%)	0.14 (−19.8%)
F1-Score	0.20	0.21 (+4.4%)	0.18 (−16.4%)
Accuracy	0.61	0.63 (+3.3%)	0.63 (0.0%)

The absolute metric values are modest because OA is only one algorithm for detecting one class of interferences, and the ground-truth labels capture all interferences globally, including those beyond OA’s scope. The meaningful signal for depth-limit tuning lies in the *shifts* in metrics across configurations.

From depth 5 to depth 10, precision improves slightly while recall remains flat. The precision gain is driven by the time-out explosion effect: false-positive scenarios that previously completed now exhaust their budget and exit the detected set, reducing reported positives without losing any true ones. Recall stays unchanged because the additional write sites reached at greater depth do not correspond to ground-truth interferences, consistent with the partial-capture nature of iOA. F1 therefore improves marginally, reflecting the precision shift rather than any genuine gain in interference detection quality.

Going from depth 10 to depth 20, both precision and recall degrade. Precision does not continue improving because the timeout explosion now also claims a true positive: a scenario with a genuine interference times out before that interference is reached, the classic outcome where a sibling subtree exhausts the budget before the relevant write site is visited. Recall falls as a result of this true positive loss. F1 reaches its lowest point across the three configurations, and accuracy remains stable, masking the opposing TP and FP shifts that cancel out in the aggregate count.

Taken together, RQ1 and RQ2 paint a consistent picture: moving from depth 5 to depth 10 yields a net gain in total interferences detected and a modest precision improvement, but the recall is unchanged and the accuracy gain is an artifact of timeout-driven FP reduction rather than genuine detection improvement. Beyond depth 10, both detection and accuracy degrade. Depth 10 is therefore the better choice over depth 5, but practitioners should not expect a reliable accuracy improvement (the gain is real on average, but structurally accidental).

5.4 RQ3: Computational Performance Across Depth Configurations

To characterize the computational cost of depth tuning we focus on the *RefDataset*, which, with its 846 scenarios (after build filtering) spanning a broader range of project sizes and merge patterns, provides a more representative and diverse workload than the *MergeDataset* for assessing performance behavior at scale.

Table 8: Execution time analysis for *RefDataset* per depth: total time and per-execution statistics (s).

Depth	Total (s)	Avg	Med.	75th	90th	95th	99th	Max
5	7,624	6.51	2.52	4.14	7.12	12.88	116.41	551.04
10	23,784	22.59	2.63	4.59	10.37	45.47	537.74	1200.22
20	55,885	54.24	2.62	4.62	14.04	143.27	1200.04	1200.11

Aggregate cost grows super-linearly: depth 10 requires $3.1\times$ as much time as depth 5, and depth 20 requires $7.3\times$ as much, well above the $4\times$ expected under linear scaling with depth. The steepest unit increase falls in the $5\rightarrow 10$ transition, exactly the step with the largest detection gain per RQ1. The median execution time is virtually unchanged across depths (2.52s, 2.63s, and 2.62s), while the mean climbs from 6.5s to 22.6s to 54.2s. This divergence reveals a bimodal execution profile: most runs complete in seconds regardless of depth, while a growing tail of pathological runs, caused by timeout explosion (Section 3), drives the aggregate upward. At depth 20, both the 99th-percentile (1200.04s) and the maximum (1200.11s) coincide with the timeout cap, confirming that at least 1% of runs exhaust the budget. Because the *RefDataset* uses the *partial-result strategy*, these runs still produce output and are not recorded as hard timeouts, though their full cost is reflected in the distribution. Tables 9 and 10 show how memory and CPU utilization are distributed across wall time (elapsed real time), exposing the resource regime behind this cost growth.

Table 9: Memory working-set distribution (% of wall time) per depth.

Depth	0–4 GB	4–8 GB	8–12 GB	≥ 12 GB
5	5.8%	53.1%	40.3%	0.8%
10	1.7%	6.3%	31.0%	61.0%
20	0.1%	9.3%	51.2%	39.4%

Table 10: CPU utilization distribution (% of wall time) per depth.

Depth	0–30%	30–50%	50–70%	70–100%
5	5.0%	37.5%	27.9%	29.6%
10	50.2%	33.3%	13.9%	2.6%
20	74.0%	20.3%	4.6%	1.1%

At depth 5, the analysis spends most of its wall time (53.1%) in the 4–8 GB range and virtually none above 12 GB (0.8%). This modest footprint reflects the limited scope of call-graph construction at shallow depth: the traversal visits its few callee chains, builds a small call graph, allocates few pointer-analysis nodes, and requires minimal JVM stack space per recursive call. Increasing depth to 10 forces a deeper exploration: each additional level multiplies the set of reachable callees and their associated pointer sets, inflating the in-memory representation of the call structure. The effect is decisive: 61.0% of wall time moves above 12 GB, and the 4–8 GB bracket nearly vanishes (6.3%). At depth 20 this extreme saturation partially retreats to 39.4% above 12 GB, but not because pressure eases; rather, the most memory-hungry scenarios hit the timeout cap before their call graphs fully expand and never reach their worst-case footprint.

The CPU distribution tells the complementary story. At depth 5, the CPU is productive for most of the run: 57.5% of wall time exceeds 50% utilization and 29.6% exceeds 70%, the signature of a *compute-bound* workload where the call graph fits comfortably in memory and the processor spends most of its time performing analysis. As depth increases to 10, the picture inverts sharply: 50.2% of wall time falls in the 0–30% CPU range and the 70–100% bracket nearly disappears (2.6%). By depth 20, 74.0% of execution time sits below 30% CPU. This inversion (more total time, far less CPU activity) is the hallmark of a *memory-bound* regime: as deeper call-graph traversal pushes the pointer-analysis working set beyond the cache hierarchy and toward RAM saturation, the processor idles waiting on memory accesses, garbage collection, and object management rather than performing productive computation.

These findings paint a consistent picture of the performance cost of depth tuning. At depth 5, the analysis is lightweight and compute-bound: most scenarios finish in seconds, CPU utilization is high, and memory stays in the 4–8 GB range. The jump to depth 10 crosses a threshold: call-graph expansion drives memory into saturation, CPU efficiency falls, and total time multiplies by $3.1\times$, yet this step also yields the greatest detection gain per RQ1, making it the best cost-to-benefit trade-off. Beyond depth 10, cost more than doubles again ($7.3\times$ over depth 5) as the analysis enters a CPU-idle, memory-saturated regime where timeout explosion actively silences detections rather than enabling them. Depth 10 therefore best amortizes performance overhead against detection gain: the majority of scenarios complete within seconds, the memory working set is bounded, and nearly all detectable interferences are within reach.

6 Study Implications

The combined findings of RQ1, RQ2, and RQ3 form a coherent picture with concrete implications for safer code integration. Developers who merge code daily often have limited visibility into how their changes interact with concurrent modifications made by colleagues: an undetected behavioral interference can silently introduce bugs that pass textual review

and reach production. Properly configured static analysis lowers this risk, giving teams more confidence that integrated code behaves as intended. The findings below translate our empirical results into guidance for practitioners deploying iOA and for researchers building on it.

Depth 10 is the practical optimal value. Across all three research questions, depth 10 consistently emerges as the most balanced configuration. From a detection standpoint (RQ1), moving from depth 5 to depth 10 reduces the share of entirely missed *RefDataset* scenarios from 14.6% to 3.4%, and eliminates scenario loss entirely in the *MergeDataset*, meaning nearly all reachable interferences fall within 10 levels of call-chain traversal. From an accuracy standpoint (RQ2), precision improves from 0.25 to 0.28 while recall remains stable, yielding the highest F1-Score (0.21) across the three depths. From a performance standpoint (RQ3), depth 10 multiplies total execution time by $3.1\times$ relative to depth 5, a meaningful increase, yet one that still keeps most scenarios in a compute-bound regime where the analysis finishes in seconds. Practitioners configuring iOA for new codebases can therefore treat depth 10 as a principled default, revisiting it only if project-specific evidence points to a different call-chain structure.

Timeout explosion is a first-class design concern. A recurring mechanism across all three RQs is timeout explosion, the phenomenon where a single deep call subtree exhausts the per-scenario time budget before sibling subtrees, which may contain actual interferences, are ever visited. At depth 20, this effect reduces detected scenarios and lowers both precision and recall relative to depth 10, turning increased depth into a net liability rather than a benefit. Crucially, a timeout-silenced scenario produces no output, so its absence from the result set cannot be distinguished from a genuine absence of interferences. Tool designers should treat the timeout as a configurable, first-class parameter rather than a safety valve, and document clearly that silence under timeout pressure does not imply safety.

Partial results are better than no results. Our adoption of a *partial-result strategy* (capping the analysis at the timeout boundary and recording whatever the traversal reached) proved its value: scenarios that exceeded the time limit still contributed output to RQ1 and RQ3, and their partial interference sets were used in the RQ2 accuracy evaluation. Discarding such scenarios entirely, as prior implementations did, would have systematically under-reported detection rates and distorted performance distributions. We recommend that future implementations of interprocedural static analyses adopt analogous partial-result semantics and expose the completeness degree of each result, allowing downstream users to reason about uncertainty rather than treating all outputs as equally complete.

Variant selection interacts with depth tuning. All findings in this study are specific to the PA variant of OA (the pointer-analysis-based variant using SPARK, as opposed to the CHA-based noPA variant [2]). Depth tuning can only

recover interferences that the chosen call-graph algorithm makes visible in the first place; if call edges are trimmed by SPARK’s open-world assumption, no depth value will surface the corresponding interferences. Practitioners and researchers should therefore treat call-graph algorithm selection and depth-limit tuning as joint decisions, and evaluate the cost-benefit trade-off of each combination rather than optimising depth in isolation.

7 Threats to Validity

We organise threats following the four-category taxonomy of construct, internal, external, and conclusion validity [25].

Construct Validity. Our precision and recall use the LOI criterion (defined in Section 4.2), which captures locally visible interference but may miss semantically equivalent writes through aliased references or cross-method calls. Reported metrics reflect OA’s partial scope; our findings focus on *relative* shifts across depths rather than absolute values. Ground truth labels were produced via double-blind labelling.

All experiments use the PA variant of OA, which relies on SPARK pointer analysis. Barbosa et al. [2] showed that the PA variant achieves lower recall than the noPA/CHA variant: SPARK’s open-world assumption trims call edges, silencing detections even at identical depth settings. Our depth-tuning conclusions therefore characterise SPARK-based OA specifically and may not generalise to the CHA-based variant.

Internal Validity. Depth and timeout interact as confounders: complex call graphs (likely at depth 20) risk timeout before full exploration. Our *partial-result strategy* makes depth 20 detections a lower bound on unconstrained analysis. SPARK’s unsoundness with Java reflection may cause missed interferences in reflection-heavy code.

External Validity. Scenarios are GitHub Java projects filtered by build success and textual conflicts, favouring well-maintained code; results may not transfer to frameworks or reflection-heavy projects. The same-method filter over-represents interference-prone scenarios, potentially inflating precision and recall compared to real-world distributions [3]. Only three depth values were evaluated, leaving precise inflection points unlocated. MergeDataset’s 99 units from limited projects constrains statistical power; RefDataset lacks per-scenario ground truth.

Conclusion Validity. Small true-positive counts on MergeDataset make precision, recall, and F1 sensitive to individual outcomes. No formal significance tests were applied; differences are descriptive. F1 comparisons should be interpreted as directional.

8 Related Work

The challenges of collaborative software development and code integration have been extensively studied. Ghiotto et al. [10] investigated the nature of merge conflicts across thousands of open-source projects, categorizing the types of textual conflicts developers face. Similarly, Elias et al. [8]

proposed approaches to recommend resolution strategies for such conflicts, and Campos Junior et al. [6] investigated how code composition strategies and line-combination patterns affect manual merge conflict resolution. While these studies provide valuable insights into textual conflicts and their resolution, our work focuses specifically on dynamic semantic conflicts, behavioral interferences that occur even when textual merges succeed without warnings.

To detect such semantic conflicts, some approaches rely on testing. Da Silva et al. [21] proposed an automated behavior change detection approach using unit test generation to identify semantic conflicts. Unlike test-based methods, which are inherently limited by the quality and coverage of the project’s test suite and can be computationally expensive to run repeatedly, our study evaluates a static analysis that does not rely on the existence of tests, but instead systematically traverses the program’s call graph to find overriding assignments.

Other researchers have explored formal and heavyweight static analysis. Horwitz et al. [11] introduced the use of Program Dependency Graphs (PDGs) and System Dependency Graphs (SDGs) for integrating non-interfering program versions. Sousa et al. [23] proposed SafeMerge, a formal verification approach to guarantee that three-way merges do not introduce unwanted behaviors. Although theoretically sound and providing formal guarantees, these techniques often require building complete dependency graphs or formal models, struggling to scale to large, real-world codebases. In contrast, our work builds upon the lightweight Overriding Assignment (OA) analysis [3, 7], which performs a targeted scan directly on the merged code. Instead of focusing on formal verification, we empirically investigate how tuning the traversal depth limit of this lightweight analysis impacts its detectability and performance.

The most closely related work to ours is by Barbosa et al. [2], who investigated the effect of different call-graph construction algorithms on the accuracy and execution time of the OA analysis. While they focused on the precision of the underlying call graph, they kept the method traversal depth fixed at a default limit of 5. Our study directly complements their work by fixing the call-graph algorithm to the most prominent configuration they evaluated (SPARK) and systematically exploring the orthogonal dimension of traversal depth. Together, these studies provide a comprehensive empirical understanding of how to configure lightweight static analysis for practical semantic conflict detection.

The broader literature on static analysis configuration [14, 15, 22, 24] provides important context for the depth-tuning problem. Prior studies extensively document the precision-scalability tradeoff inherent in parameters like context sensitivity, which is analogous to our traversal depth limit. They generally show that while deeper context improves precision, it does so monotonically until the computational cost becomes prohibitive, often requiring selective optimization to maintain scalability. Our study contrasts with this classical view by applying depth tuning specifically to semantic merge conflict detection. In this domain, we reveal a non-monotonic

detectability-cost relationship driven by timeout explosion, a dynamic rarely highlighted in general static analysis literature, since those analyses typically do not operate under the strict, per-scenario time budgets required for practical merge interference detection.

9 Conclusion

This paper presented an empirical investigation into how varying the call-graph traversal depth limit (set to 5, 10, and 20) affects both the detectability and the computational cost of the Overriding Assignment (OA) static analysis for semantic merge conflict detection. Evaluating on two datasets of Java open-source merge scenarios, we found that the relationship between depth and detection is non-monotonic: moving from depth 5 to depth 10 reduces the share of entirely missed scenarios and yields the highest accuracy score on the *MergeDataset* (tied with depth 20), while pushing depth to 20 reverses these gains as timeout explosion silences detections before they are reached. On the performance side, total execution time grows super-linearly ($3.1\times$ at depth 10, $7.3\times$ at depth 20 over depth 5), and the resource regime shifts from compute-bound to memory-bound as deeper call-graph traversal drives the pointer-analysis working set into RAM saturation. Together, these findings establish depth 10 as a principled default that best amortises detection gain against computational cost for the configurations and datasets studied.

Artifact Availability

To support the replication of our study and encourage further research, we provide all datasets, scripts, and experimental results in an online appendix [1].

References

- [1] 2026. Online Appendix. available at: <https://anonymous.4open.science/w/papers-AEC8/oa-deep-sbes26.html>.
- [2] Matheus Barbosa, Paulo Borba, Rodrigo Bonifácio, Victor Lira, and Galileu Santos. 2025. The Effect of Pointer Analysis on Semantic Conflict Detection. *arXiv preprint arXiv:2507.20081* (2025). <https://arxiv.org/abs/2507.20081>
- [3] Matheus Barbosa, Paulo Borba, Rodrigo Bonifacio, and Galileu Santos. 2022. Semantic Conflict Detection with Overriding Assignment Analysis. In *Proceedings of the XXXVI Brazilian Symposium on Software Engineering (SBES 2022)*. ACM, New York, NY, USA, 435–445. doi:10.1145/3555228.3555242
- [4] Roberto Souto Maior de BARROS FILHO. 2017. *Using information flow to estimate interference between same-method contributions*. Master’s thesis. Universidade Federal de Pernambuco.
- [5] Christian Bird and Thomas Zimmermann. 2012. Assessing the Value of Branches with What-if Analysis. In *Proceedings of the 20th International Symposium on Foundations of Software Engineering (FSE 2012)*. ACM, New York, NY, USA. <https://www.microsoft.com/en-us/research/publication/assessing-the-value-of-branches-with-what-if-analysis/>
- [6] Heleno de S. Campos Junior, Gleiph Giotto L. de Menezes, Márcio de Oliveira Barros, André van der Hoek, and Leonardo Gresta Paulino Murta. 2024. How code composition strategies affect merge conflict resolution? *Journal of Software Engineering Research and Development* 12, 1 (Oct. 2024), 13:1 – 13:17. doi:10.5753/jserd.2024.3638
- [7] Galileu Santos de Jesus, Paulo Borba, Rodrigo Bonifácio, and Matheus Barbosa. 2024. Lightweight Semantic Conflict Detection with Static Analysis. In *Companion Proceedings of the 46th*

- International Conference on Software Engineering (ICSE 2024)*. ACM, New York, NY, USA, 340–343.
- [8] Paulo Elias, Heleno de S. Campos, Eduardo Ogasawara, and Leonardo Gresta Paulino Murta. 2023. Towards accurate recommendations of merge conflicts resolution strategies. *Information and Software Technology* 164 (2023), 107332. doi:10.1016/j.infsof.2023.107332
 - [9] H. Christian Estler, Martin Nordio, Carlo A. Furia, and Bertrand Meyer. 2014. Awareness and Merge Conflicts in Distributed Software Development. In *2014 IEEE 9th International Conference on Global Software Engineering (ICGSE)*. 26–35. doi:10.1109/ICGSE.2014.17
 - [10] Gleiph Ghiotto, Leonardo Murta, Márcio Barros, and Andre Van Der Hoek. 2018. On the nature of merge conflicts: a study of 2,731 open source java projects hosted by github. *IEEE Transactions on Software Engineering* (2018), 892–915.
 - [11] Susan Horwitz, Jan Prins, and Thomas Reps. 1989. Integrating Noninterfering Versions of Programs. *ACM Transactions on Programming Languages and Systems* 11, 3 (1989), 345–387. doi:10.1145/65979.65980
 - [12] Sanjeev Khanna, Keshav Kunal, and Benjamin C. Pierce. 2007. A Formal Investigation of Diff3. In *Proceedings of the 27th International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2007)*. Springer-Verlag, Berlin, Heidelberg, 485–496.
 - [13] Ondřej Lhoták and Laurie Hendren. 2003. Scaling Java points-to analysis using SPARK. In *International Conference on Compiler Construction*. 153–169.
 - [14] Ondřej Lhoták and Laurie Hendren. 2006. Context-Sensitive Points-to Analysis: Is It Worth It?. In *Proceedings of the 15th International Conference on Compiler Construction (CC 2006)*. Springer, 63–78. doi:10.1007/11688839_5
 - [15] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. 2020. A Principled Approach to Selective Context Sensitivity for Pointer Analysis. *ACM Transactions on Programming Languages and Systems* 42, 2 (2020), 10:1–10:40. doi:10.1145/3381915
 - [16] Victor Lira, Paulo Borba, Rodrigo Bonifácio, Galileu Santos, and Matheus Barbosa. 2025. RefFilter: Improving Semantic Conflict Detection via Refactoring-Aware Static Analysis. *arXiv preprint arXiv:2510.01960* (2025). <https://arxiv.org/abs/2510.01960>
 - [17] Wardah Mahmood, Moses Chagama, Thorsten Berger, and Regina Hebig. 2020. Causes of merge conflicts: a case study of ElasticSearch. *arXiv preprint arXiv:2010.01960* (2020). <https://arxiv.org/abs/2010.01960>
 - [18] Shane McKee, Nicholas Nelson, Anita Sarma, and Danny Dig. 2017. Software Practitioner Perspectives on Merge Conflicts and Resolutions. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 467–478. doi:10.1109/ICSME.2017.53
 - [19] Dewayne E. Perry, Harvey P. Siy, and Lawrence G. Votta. 2001. Parallel Changes in Large-Scale Software Development: An Observational Case Study. *ACM Transactions on Software Engineering and Methodology* 10, 3 (2001), 308–337. doi:10.1145/383876.383878
 - [20] Galileu Santos de Jesus, Paulo Borba, Rodrigo Bonifácio, and Matheus Barbosa de Oliveira. 2025. Comparing static analyses for improved semantic conflict detection. *Automated Software Engineering* 33, 2 (2025), 1–38. <https://doi.org/10.1007/s10515-025-00580-y>
 - [21] Leuson Da Silva, Paulo Borba, Wardah Mahmood, Thorsten Berger, and João Moisakis. 2020. Detecting Semantic Conflicts via Automated Behavior Change Detection. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 174–184. doi:10.1109/ICSME46990.2020.00026
 - [22] Yannis Smaragdakis, Martin Bravenboer, and Ondřej Lhoták. 2011. Pick Your Contexts Well: Understanding Object-Sensitivity. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2011)*. ACM, 17–30. doi:10.1145/1926385.1926390
 - [23] Marcelo Sousa, Isil Dillig, and Shuvendu K. Lahiri. 2018. Verified Three-Way Program Merge. In *Proceedings of the ACM on Programming Languages (OOPSLA 2018)*, Vol. 2. 1–29. doi:10.1145/3276535
 - [24] Frank Tip and Jens Palsberg. 2000. Scalable Propagation-Based Call Graph Construction Algorithms. In *Proceedings of the 15th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2000)*. ACM, 281–293. doi:10.1145/345513.345657
 - [25] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer Science & Business Media.